

Metoda optymalizacji

Paweł Drzyzga
Wydział Informatyki i Matematyki
Politechnika Krakowska

19 grudnia 2025

Spis treści

1	Metody optymalizacji	3
1.1	Metoda Hooke’a–Jeevesa	3
1.1.1	Opis matematyczny algorytmu	3
1.1.2	Przykład liczbowy	3
1.1.3	Wynik końcowy	4
1.1.4	Pseudokod	4
1.1.5	Uwagi końcowe	4
1.2	Metoda Neldera–Meada (simpleks)	4
1.2.1	Opis matematyczny algorytmu	5
1.2.2	Przykład liczbowy	5
1.2.3	Kryterium zakończenia	6
1.2.4	Wynik końcowy	6
1.2.5	Pseudokod	6
1.2.6	Uwagi końcowe	7
1.3	Metoda Powella	7
1.3.1	Opis matematyczny algorytmu	7
1.3.2	Przykład liczbowy	7
1.3.3	Wynik końcowy	8
1.3.4	Pseudokod	8
1.3.5	Uwagi końcowe	8
1.4	Metoda Fletcher–Reevesa	8
1.4.1	Opis matematyczny algorytmu	9
1.4.2	Przykład liczbowy	9
1.4.3	Wynik końcowy	10
1.4.4	Pseudokod	10
1.4.5	Uwagi końcowe	10
1.5	Metoda DFP (Davidon–Fletcher–Powell)	10
1.5.1	Opis matematyczny algorytmu	10
1.5.2	Przykład liczbowy	11
1.5.3	Wynik końcowy	11
1.5.4	Pseudokod	12
1.5.5	Uwagi końcowe	12
1.6	Metoda BFGS (Broyden–Fletcher–Goldfarb–Shanno)	12
1.6.1	Opis matematyczny algorytmu	12
1.6.2	Przykład liczbowy	13
1.6.3	Wynik końcowy	13
1.6.4	Pseudokod	14
1.6.5	Uwagi końcowe	14
1.7	Algorytm ewolucyjny (GA)	14
1.7.1	Opis matematyczny algorytmu	14
1.7.2	Przykład liczbowy	15
1.7.3	Kryterium zakończenia	15
1.7.4	Wynik końcowy	15
1.7.5	Pseudokod	16
1.7.6	Uwagi końcowe	16
1.8	Instrukcja obsługi	16
1.8.1	Elementy interfejsu	16

1.8.2	Procedura użycia	17
1.8.3	Uwagi praktyczne	18

1 Metody optymalizacji

1.1 Metoda Hooke'a–Jeevesa

Metoda Hooke'a–Jeevesa jest bezgradientową, iteracyjną metodą optymalizacji, przeznaczoną do wyznaczania minimum funkcji

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}.$$

Algorytm opiera się wyłącznie na porównywaniu wartości funkcji w punktach sąsiadujących z aktualnym przybliżeniem i nie wymaga obliczania pochodnych.

1.1.1 Opis matematyczny algorytmu

Niech $x^{(k)} = (x_1^{(k)}, x_2^{(k)})$ oznacza aktualny punkt w k -tej iteracji, a $\delta > 0$ — aktualny krok przeszukiwania. W algorytmie wykorzystywane są wektory bazowe przestrzeni $\mathbb{R}^2$:

$$e_1 = (1, 0), \quad e_2 = (0, 1).$$

Jedna iteracja metody składa się z następujących etapów:

1. **Krok eksploracyjny.** Dla każdej współrzędnej sprawdzane są punkty:

$$x^{(k)} \pm \delta e_i, \quad i = 1, 2.$$

Jeżeli w którymkolwiek z tych punktów wartość funkcji jest mniejsza niż $f(x^{(k)})$, punkt zostaje zaktualizowany.

2. **Ruch wzorcowy.** Jeżeli krok eksploracyjny zakończył się poprawą, wykonywany jest ruch wzorcowy:

$$x_{\text{pattern}} = x_{\text{new}} + \alpha(x_{\text{new}} - x^{(k)}),$$

gdzie $\alpha > 1$ jest ustalonym parametrem (w implementacji przyjęto $\alpha = 2$). Jeżeli $f(x_{\text{pattern}}) < f(x_{\text{new}})$, punkt wzorcowy zostaje zaakceptowany.

3. **Redukcja kroku.** Jeżeli w kroku eksploracyjnym nie znaleziono poprawy, krok jest zmniejszany:

$$\delta \leftarrow \frac{\delta}{2}.$$

Algorytm jest wykonywany iteracyjnie aż do spełnienia kryterium stopu:

$$\delta \leq \varepsilon,$$

gdzie $\varepsilon > 0$ jest zadaną tolerancją.

1.1.2 Przykład liczbowy

Rozważmy funkcję:

$$f(x, y) = (x - 1)^2 + (y - 2)^2,$$

która posiada minimum globalne w punkcie $(1, 2)$.

Dane początkowe:

$$x^{(0)} = (4, 4), \quad \delta_0 = 2, \quad \alpha = 2, \quad \varepsilon = 0.5.$$

Wartość funkcji w punkcie startowym:

$$f(4, 4) = (4 - 1)^2 + (4 - 2)^2 = 13.$$

Iteracja 1. *Krok eksploracyjny w kierunku osi x:*

$$f(6, 4) = 29, \quad f(2, 4) = 5.$$

Punkt zostaje zaktualizowany do $(2, 4)$.

Krok eksploracyjny w kierunku osi y:

$$f(2, 6) = 17, \quad f(2, 2) = 1.$$

Nowy punkt:

$$x^{(1)} = (2, 2).$$

Ruch wzorcowy:

$$x_{\text{pattern}} = (2, 2) + 2((2, 2) - (4, 4)) = (-2, -2),$$

$$f(-2, -2) = 25.$$

Ruch wzorcowy zostaje odrzucony.

Iteracja 2. Dla $\delta = 2$ sprawdzane punkty:

$$(4, 2), (0, 2), (2, 4), (2, 0),$$

nie dają poprawy wartości funkcji. W związku z tym krok zostaje zredukowany:

$$\delta \leftarrow 1.$$

Iteracja 3. *Krok eksploracyjny w kierunku osi x :*

$$f(3, 2) = 4, \quad f(1, 2) = 0.$$

Punkt zostaje zaktualizowany do $(1, 2)$.

Krok eksploracyjny w kierunku osi y :

$$f(1, 3) = 1, \quad f(1, 1) = 1.$$

Brak dalszej poprawy.

1.1.3 Wynik końcowy

Po trzech iteracjach algorytmu otrzymano punkt:

$$x^* = (1, 2),$$

który jest minimum globalnym analizowanej funkcji.

1.1.4 Pseudokod

Algorithm 1 Metoda poszukiwań wzorcowych (Hooke–Jeeves)

```
1: Dane: funkcja  $f(x)$ , punkt startowy  $x_0$ , krok początkowy  $\delta_0$ , tolerancja  $\varepsilon$ 
2: Wynik: przybliżone minimum  $x^*$ 
3:  $x \leftarrow x_0$ 
4:  $\delta \leftarrow \delta_0$ 
5: while  $\delta > \varepsilon$  do
6:    $x_{\text{new}} \leftarrow x$ 
7:   for każda współrzędna  $i$  do
8:     if  $f(x_{\text{new}} + \delta e_i) < f(x_{\text{new}})$  then
9:        $x_{\text{new}} \leftarrow x_{\text{new}} + \delta e_i$ 
10:    else if  $f(x_{\text{new}} - \delta e_i) < f(x_{\text{new}})$  then
11:       $x_{\text{new}} \leftarrow x_{\text{new}} - \delta e_i$ 
12:    end if
13:  end for
14:  if  $f(x_{\text{new}}) < f(x)$  then
15:     $x \leftarrow x + (x_{\text{new}} - x)$  {ruch wzorcowy}
16:  else
17:     $\delta \leftarrow \delta/2$ 
18:  end if
19: end while
20:  $x^* \leftarrow x$ 
```

1.1.5 Uwagi końcowe

Metoda Hooke’a–Jeevesa wykorzystuje wyłącznie porównania wartości funkcji oraz proste operacje arytmetyczne. Dzięki temu jest odporna na brak gładkości funkcji, jednak jej zbieżność może być wolniejsza w porównaniu z metodami gradientowymi.

1.2 Metoda Neldera–Meada (simpleks)

Metoda Neldera–Meada jest bezgradientową, iteracyjną metodą optymalizacji przeznaczoną do minimalizacji funkcji

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}.$$

Algorytm nie operuje na pojedynczym punkcie, lecz na zbiorze punktów tworzących *simpleks*. W przestrzeni dwuwymiarowej simpleks składa się z trzech wierzchołków.

Metoda wykorzystuje jedynie wartości funkcji w wierzchołkach simpleksu oraz proste operacje liniowe na wektorach, bez konieczności obliczania pochodnych.

1.2.1 Opis matematyczny algorytmu

Niech

$$S^{(k)} = \{x_1^{(k)}, x_2^{(k)}, x_3^{(k)}\}$$

oznacza simpleks w k -tej iteracji. W każdej iteracji wierzchołki są sortowane według wartości funkcji:

$$f(x_B^{(k)}) \leq f(x_G^{(k)}) \leq f(x_W^{(k)}),$$

gdzie:

- $x_B^{(k)}$ — najlepszy punkt,
- $x_G^{(k)}$ — punkt pośredni,
- $x_W^{(k)}$ — najgorszy punkt.

Centroid simpleksu (z pominięciem najgorszego punktu) dany jest wzorem:

$$c^{(k)} = \frac{1}{2}(x_B^{(k)} + x_G^{(k)}).$$

Na tej podstawie wyznaczane są kolejne punkty testowe:

• **Odbicie:**

$$x_R = c^{(k)} + \alpha(c^{(k)} - x_W^{(k)}),$$

• **Ekspansja:**

$$x_E = c^{(k)} + \gamma(x_R - c^{(k)}),$$

• **Kontrakcja:**

$$x_C = c^{(k)} + \frac{1}{2}(x_W^{(k)} - c^{(k)}),$$

gdzie $\alpha > 0$ oraz $\gamma > 1$ są ustalonymi parametrami algorytmu (w implementacji przyjęto $\alpha = 1$, $\gamma = 2$).

W zależności od wartości funkcji w punktach x_R , x_E oraz x_C , najgorszy wierzchołek simpleksu zostaje zastąpiony odpowiednim punktem.

1.2.2 Przykład liczbowy

Rozważmy funkcję:

$$f(x, y) = (x - 1)^2 + (y - 2)^2,$$

która posiada minimum globalne w punkcie $(1, 2)$.

Jako simpleks początkowy przyjmujemy:

$$x_1^{(0)} = (4, 4), \quad x_2^{(0)} = (6, 4), \quad x_3^{(0)} = (4, 6).$$

Wartości funkcji:

$$f(4, 4) = 13, \quad f(6, 4) = 29, \quad f(4, 6) = 25.$$

Iteracja 1. Po posortowaniu wierzchołków:

$$x_B = (4, 4), \quad x_G = (4, 6), \quad x_W = (6, 4).$$

Centroid:

$$c = \frac{(4, 4) + (4, 6)}{2} = (4, 5).$$

Odbicie:

$$x_R = 2c - x_W = (2, 6), \quad f(2, 6) = 17.$$

Ponieważ $f(x_B) < f(x_R) < f(x_G)$, punkt x_R zostaje zaakceptowany i zastępuje x_W .

Nowy simpleks:

$$(4, 4), (4, 6), (2, 6).$$

Iteracja 2. Po sortowaniu:

$$x_B = (4, 4), \quad x_G = (2, 6), \quad x_W = (4, 6).$$

Centroid:

$$c = \frac{(4, 4) + (2, 6)}{2} = (3, 5).$$

Odbicie:

$$x_R = 2c - x_W = (2, 4), \quad f(2, 4) = 5.$$

Ponieważ $f(x_R) < f(x_B)$, wykonywana jest ekspansja:

$$x_E = c + 2(x_R - c) = (1, 3), \quad f(1, 3) = 1.$$

Punkt x_E zostaje zaakceptowany.

Iteracja 3. Simpleks:

$$(4, 4), (2, 6), (1, 3).$$

Po dalszych operacjach odbicia i kontrakcji simpleks ulega stopniowemu zmniejszeniu i koncentruje się w otoczeniu punktu $(1, 2)$.

1.2.3 Kryterium zakończenia

Algorytm Nelder–Meada kończy działanie, gdy spełnione jest jedno z kryteriów:

- rozmiar simpleksu jest mniejszy od zadanej tolerancji,
- różnice wartości funkcji w wierzchołkach simpleksu są niewielkie,
- osiągnięto maksymalną liczbę iteracji.

1.2.4 Wynik końcowy

W zaprezentowanym przykładzie algorytm zbiega do punktu:

$$x^* \approx (1, 2),$$

który jest minimum globalnym analizowanej funkcji.

1.2.5 Pseudokod

Algorithm 2 Metoda Nelder–Meada (simpleksowa)

```
1: Dane: funkcja  $f(x)$ , punkt startowy  $x_0$ , tolerancja  $\varepsilon$ 
2: Wynik: minimum  $x^*$ 
3: Zainicjalizuj simpleks  $\{x_1, x_2, x_3\}$ 
4: while kryterium stopu nie jest spełnione do
5:   Posortuj wierzchołki simpleksu według wartości  $f$ 
6:   Wyznacz  $x_{\text{best}}$ ,  $x_{\text{second\_worst}}$ ,  $x_{\text{worst}}$ 
7:   Wyznacz środek ciężkości  $c$  bez najgorszego punktu:
8:      $c \leftarrow \frac{1}{2}(x_{\text{best}} + x_{\text{second\_worst}})$ 
9:   Odbicie:  $x_r \leftarrow c + \alpha(c - x_{\text{worst}})$ 
10:  if  $f(x_{\text{best}}) \leq f(x_r)$  and  $f(x_r) < f(x_{\text{second\_worst}})$  then
11:    Zastąp  $x_{\text{worst}} \leftarrow x_r$ 
12:  else
13:    if  $f(x_r) < f(x_{\text{best}})$  then
14:      Ekspansja:  $x_e \leftarrow c + \gamma(x_r - c)$ 
15:      if  $f(x_e) < f(x_r)$  then
16:        Zastąp  $x_{\text{worst}} \leftarrow x_e$ 
17:      else
18:        Zastąp  $x_{\text{worst}} \leftarrow x_r$ 
19:      end if
20:    else
21:      Wykonaj kontrakcję albo redukcję simpleksu
22:    end if
23:  end if
24: end while
25:  $x^* \leftarrow x_{\text{best}}$ 
```

1.2.6 Uwagi końcowe

Metoda Neldera–Meada jest szczególnie użyteczna w przypadku funkcji, dla których obliczanie pochodnych jest trudne lub niemożliwe. Jej wadą jest brak gwarancji zbieżności do minimum globalnego w przypadku funkcji o złożonej strukturze.

1.3 Metoda Powella

Metoda Powella jest bezgradientową, iteracyjną metodą optymalizacji, przeznaczoną do minimalizacji funkcji

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}.$$

Algorytm polega na wykonywaniu kolejnych minimalizacji jednowymiarowych wzdłuż zadanych kierunków, które są aktualizowane w trakcie działania algorytmu. Metoda nie wymaga obliczania pochodnych funkcji.

1.3.1 Opis matematyczny algorytmu

Niech $x^{(k)} \in \mathbb{R}^2$ oznacza punkt w k -tej iteracji algorytmu, a zbiór kierunków:

$$D^{(k)} = \{d_1^{(k)}, d_2^{(k)}\}$$

stanowi bazę kierunków przeszukiwania.

Początkowo przyjmuje się:

$$d_1^{(0)} = (1, 0), \quad d_2^{(0)} = (0, 1).$$

Jedna iteracja metody Powella składa się z następujących etapów:

1. Minimalizacja wzdłuż kierunku $d_1^{(k)}$:

$$x \leftarrow \arg \min_{\alpha} f(x^{(k)} + \alpha d_1^{(k)}).$$

2. Minimalizacja wzdłuż kierunku $d_2^{(k)}$:

$$x \leftarrow \arg \min_{\alpha} f(x + \alpha d_2^{(k)}).$$

3. Wyznaczenie nowego kierunku:

$$d_{\text{new}} = x - x^{(k)}.$$

4. Aktualizacja zbioru kierunków:

$$D^{(k+1)} = \{d_2^{(k)}, d_{\text{new}}\}.$$

Algorytm jest wykonywany iteracyjnie aż do spełnienia kryterium stopu:

$$\|x^{(k+1)} - x^{(k)}\| < \varepsilon,$$

gdzie $\varepsilon > 0$ jest zadaną tolerancją.

1.3.2 Przykład liczbowy

Rozważmy funkcję:

$$f(x, y) = (x - 1)^2 + (y - 2)^2,$$

która posiada minimum globalne w punkcie $(1, 2)$.

Dane początkowe:

$$x^{(0)} = (4, 4), \quad d_1^{(0)} = (1, 0), \quad d_2^{(0)} = (0, 1).$$

Iteracja 1. Minimalizacja wzdłuż kierunku $d_1^{(0)} = (1, 0)$:

Rozważamy funkcję jednej zmiennej:

$$\varphi_1(\alpha) = f(4 + \alpha, 4) = (3 + \alpha)^2 + 4.$$

Minimum osiągane jest dla:

$$\alpha^* = -3.$$

Nowy punkt:

$$x = (4, 4) + (-3)(1, 0) = (1, 4).$$

Minimalizacja wzdłuż kierunku $d_2^{(0)} = (0, 1)$:

$$\varphi_2(\alpha) = f(1, 4 + \alpha) = (2 + \alpha)^2.$$

Minimum osiągane jest dla:

$$\alpha^* = -2.$$

Nowy punkt:

$$x^{(1)} = (1, 2).$$

Wyznaczenie nowego kierunku:

$$d_{\text{new}} = x^{(1)} - x^{(0)} = (-3, -2).$$

Zaktualizowany zbiór kierunków:

$$d_1^{(1)} = (0, 1), \quad d_2^{(1)} = (-3, -2).$$

Iteracja 2. Minimalizacja wzdłuż kierunku $d_1^{(1)} = (0, 1)$:

$$\varphi_1(\alpha) = f(1, 2 + \alpha) = \alpha^2,$$

co prowadzi do minimum dla $\alpha = 0$.

Minimalizacja wzdłuż kierunku $d_2^{(1)} = (-3, -2)$:

$$\varphi_2(\alpha) = f(1 - 3\alpha, 2 - 2\alpha) = 13\alpha^2,$$

której minimum również osiągane jest dla $\alpha = 0$.

Ponieważ punkt nie ulega zmianie, algorytm spełnia kryterium stopu.

1.3.3 Wynik końcowy

Metoda Powella zakończyła działanie po dwóch iteracjach, zwracając punkt:

$$x^* = (1, 2),$$

który jest minimum globalnym analizowanej funkcji.

1.3.4 Pseudokod

Algorithm 3 Metoda Powella (bezpochodna, z przeszukiwaniem liniowym)

```
1: Dane: funkcja  $f(x)$ , punkt startowy  $x_0$ , tolerancja  $\varepsilon$ 
2: Wynik: minimum  $x^*$ 
3: Ustal zbiór kierunków  $D = \{e_1, e_2\}$ 
4:  $x \leftarrow x_0$ 
5: while nie spełniono kryterium stopu do
6:    $x_{\text{start}} \leftarrow x$ 
7:   for każdy kierunek  $d \in D$  do
8:     Wyznacz
9:      $\alpha^* \leftarrow \arg \min_{\alpha} f(x + \alpha d)$ 
10:     $x \leftarrow x + \alpha^* d$ 
11:   end for
12:    $d_{\text{new}} \leftarrow x - x_{\text{start}}$ 
13:   Zastąp jeden z kierunków w  $D$  przez  $d_{\text{new}}$ 
14: end while
15:  $x^* \leftarrow x$ 
```

1.3.5 Uwagi końcowe

Metoda Powella jest skuteczna w przypadku funkcji gładkich i niskowymiarowych. Jej zaletą jest brak konieczności obliczania gradientów, natomiast wadą może być szybki wzrost kosztu obliczeń dla funkcji o wyższym wymiarze.

1.4 Metoda Fletcher–Reevesa

Metoda Fletcher–Reevesa należy do klasy gradientowych metod iteracyjnych i stanowi wariant metody sprzężonych gradientów. Jej celem jest minimalizacja funkcji

$$f : \mathbb{R}^2 \rightarrow \mathbb{R},$$

przy wykorzystaniu informacji o gradiencie funkcji oraz kierunków sprzężonych, które zapewniają szybszą zbieżność niż metoda najszybszego spadku.

1.4.1 Opis matematyczny algorytmu

Niech $x^{(k)} \in \mathbb{R}^2$ oznacza punkt w k -tej iteracji algorytmu, a gradient funkcji w tym punkcie:

$$g^{(k)} = \nabla f(x^{(k)}).$$

Kierunek poszukiwań definiowany jest jako:

$$d^{(k)} = \begin{cases} -g^{(0)}, & k = 0, \\ -g^{(k)} + \beta_k d^{(k-1)}, & k \geq 1, \end{cases}$$

gdzie współczynnik Fletcher–Reevesa dany jest wzorem:

$$\beta_k = \frac{\|g^{(k)}\|^2}{\|g^{(k-1)}\|^2}.$$

Aktualizacja punktu następuje zgodnie z zależnością:

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)},$$

gdzie $\alpha_k > 0$ jest krokiem wyznaczanym w procesie jednowymiarowej minimalizacji wzdłuż kierunku $d^{(k)}$.

Algorytm jest powtarzany aż do spełnienia kryterium stopu:

$$\|\nabla f(x^{(k)})\| < \varepsilon.$$

1.4.2 Przykład liczbowy

Rozważmy funkcję:

$$f(x, y) = x^2 + 2y^2,$$

która posiada minimum globalne w punkcie $(0, 0)$.

Gradient funkcji:

$$\nabla f(x, y) = (2x, 4y).$$

Dane początkowe:

$$x^{(0)} = (1, 1), \quad \varepsilon > 0.$$

Iteracja 0. Gradient:

$$g^{(0)} = (2, 4).$$

Kierunek:

$$d^{(0)} = -g^{(0)} = (-2, -4).$$

Rozważamy funkcję jednej zmiennej:

$$\varphi(\alpha) = f(1 - 2\alpha, 1 - 4\alpha) = 3 - 20\alpha + 36\alpha^2.$$

Minimum osiągane jest dla:

$$\alpha_0 = \frac{5}{18}.$$

Nowy punkt:

$$x^{(1)} = (1, 1) + \frac{5}{18}(-2, -4) = \left(\frac{4}{9}, -\frac{1}{9}\right).$$

Iteracja 1. Gradient:

$$g^{(1)} = \left(\frac{8}{9}, -\frac{4}{9}\right).$$

Współczynnik:

$$\beta_1 = \frac{\|g^{(1)}\|^2}{\|g^{(0)}\|^2} = \frac{80/81}{20} = \frac{4}{81}.$$

Kierunek:

$$d^{(1)} = -g^{(1)} + \beta_1 d^{(0)} = \left(-\frac{80}{81}, \frac{20}{81}\right).$$

Po wykonaniu line search otrzymujemy:

$$\alpha_1 = \frac{9}{20}.$$

Nowy punkt:

$$x^{(2)} = \left(\frac{4}{9}, -\frac{1}{9}\right) + \frac{9}{20} \left(-\frac{80}{81}, \frac{20}{81}\right) = (0, 0).$$

Iteracja 2. Gradient:

$$g^{(2)} = (0, 0).$$

Ponieważ $\|\nabla f(x^{(2)})\| = 0$, spełnione jest kryterium stopu i algorytm zostaje zakończony.

1.4.3 Wynik końcowy

Metoda Fletcher–Reevesa zakończyła działanie po trzech iteracjach, zwracając punkt:

$$x^* = (0, 0),$$

który jest minimum globalnym analizowanej funkcji.

1.4.4 Pseudokod

Algorithm 4 Metoda gradientów sprzężonych (Fletcher–Reevesa)

```
1: Dane: funkcja  $f(x)$ , punkt startowy  $x_0$ , tolerancja  $\varepsilon$ 
2: Wynik: minimum  $x^*$ 
3:  $x \leftarrow x_0$ 
4:  $g \leftarrow \nabla f(x)$ 
5:  $d \leftarrow -g$ 
6: while  $\|g\| > \varepsilon$  do
7:   Wyznacz krok  $\alpha$  (przeszukiwanie liniowe)
8:    $x_{\text{new}} \leftarrow x + \alpha d$ 
9:    $g_{\text{new}} \leftarrow \nabla f(x_{\text{new}})$ 
10:   $\beta \leftarrow \frac{g_{\text{new}}^T g_{\text{new}}}{g^T g}$ 
11:   $d \leftarrow -g_{\text{new}} + \beta d$ 
12:   $x \leftarrow x_{\text{new}}$ 
13:   $g \leftarrow g_{\text{new}}$ 
14: end while
15:  $x^* \leftarrow x$ 
```

1.4.5 Uwagi końcowe

Metoda Fletcher–Reevesa wykorzystuje informację o gradiencie funkcji oraz sprzężone kierunki poszukiwań, co pozwala na szybszą zbieżność w porównaniu z metodą najszybszego spadku. Dla funkcji kwadratowych algorytm osiąga minimum w skończonej liczbie iteracji.

1.5 Metoda DFP (Davidon–Fletcher–Powell)

Metoda DFP należy do klasy metod quasi-Newtona i służy do minimalizacji funkcji

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}$$

z wykorzystaniem informacji o gradiencie, bez konieczności jawnego obliczania macierzy Heszana. Algorytm konstruuje w kolejnych iteracjach aproksymację odwrotności Heszana, co pozwala na szybszą zbieżność niż w klasycznych metodach gradientowych.

1.5.1 Opis matematyczny algorytmu

Niech $x^{(k)} \in \mathbb{R}^2$ oznacza punkt w k -tej iteracji algorytmu, a gradient funkcji w tym punkcie:

$$g^{(k)} = \nabla f(x^{(k)}).$$

W metodzie DFP przechowywana jest macierz H_k , stanowiąca przybliżenie odwrotności Heszana $(\nabla^2 f(x^{(k)}))^{-1}$. Kierunek poszukiwań wyznaczany jest jako:

$$d^{(k)} = -H_k g^{(k)}.$$

Po wykonaniu jednowymiarowej minimalizacji wzdłuż kierunku $d^{(k)}$ otrzymuje się nowy punkt:

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}.$$

Definiujemy wektory:

$$s_k = x^{(k+1)} - x^{(k)}, \quad y_k = g^{(k+1)} - g^{(k)}.$$

Aktualizacja macierzy H_k w metodzie DFP dana jest wzorem:

$$H_{k+1} = H_k + \frac{s_k s_k^T}{s_k^T y_k} - \frac{H_k y_k y_k^T H_k}{y_k^T H_k y_k}.$$

Algorytm jest wykonywany iteracyjnie aż do spełnienia kryterium stopu:

$$\|\nabla f(x^{(k)})\| < \varepsilon.$$

1.5.2 Przykład liczbowy

Rozważmy funkcję:

$$f(x, y) = x^2 + 2y^2,$$

która posiada minimum globalne w punkcie $(0, 0)$.

Gradient funkcji:

$$\nabla f(x, y) = (2x, 4y).$$

Dane początkowe:

$$x^{(0)} = (1, 1), \quad H_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Iteracja 0. Gradient:

$$g^{(0)} = (2, 4).$$

Kierunek:

$$d^{(0)} = -H_0 g^{(0)} = (-2, -4).$$

Jednowymiarowa minimalizacja prowadzi do kroku:

$$\alpha_0 = \frac{5}{18}.$$

Nowy punkt:

$$x^{(1)} = (1, 1) + \frac{5}{18}(-2, -4) = \left(\frac{4}{9}, -\frac{1}{9}\right).$$

Gradient:

$$g^{(1)} = \left(\frac{8}{9}, -\frac{4}{9}\right).$$

Aktualizacja macierzy H_1 .

$$s_0 = \left(-\frac{5}{9}, -\frac{10}{9}\right), \quad y_0 = \left(-\frac{10}{9}, -\frac{40}{9}\right).$$

Po podstawieniu do wzoru DFP otrzymujemy nową macierz H_1 , która lepiej aproksymuje odwrotność Hesniana funkcji f .

Iteracja 1. Kierunek:

$$d^{(1)} = -H_1 g^{(1)}.$$

Po wykonaniu jednowymiarowej minimalizacji wzdłuż tego kierunku otrzymujemy punkt:

$$x^{(2)} = (0, 0).$$

Gradient:

$$g^{(2)} = (0, 0).$$

Ponieważ $\|\nabla f(x^{(2)})\| = 0$, spełnione jest kryterium stopu i algorytm zostaje zakończony.

1.5.3 Wynik końcowy

Metoda DFP zakończyła działanie po dwóch iteracjach, zwracając punkt:

$$x^* = (0, 0),$$

który jest minimum globalnym analizowanej funkcji.

1.5.4 Pseudokod

Algorithm 5 Metoda BFGS (quasi-Newtona)

```
1: Dane: funkcja  $f(x)$ , punkt startowy  $x_0$ , tolerancja  $\varepsilon$ 
2: Wynik: minimum  $x^*$ 
3:  $x \leftarrow x_0$ 
4:  $H \leftarrow I$  {początkowa aproksymacja odwrotności Hesjanu}
5: while  $\|\nabla f(x)\| > \varepsilon$  do
6:    $d \leftarrow -H\nabla f(x)$ 
7:   Wyznacz krok  $\alpha$  (przeszukiwanie liniowe)
8:    $x_{\text{new}} \leftarrow x + \alpha d$ 
9:    $s \leftarrow x_{\text{new}} - x$ 
10:   $y \leftarrow \nabla f(x_{\text{new}}) - \nabla f(x)$ 
11:   $H \leftarrow H + \frac{ss^T}{s^T y} - \frac{Hyy^T H}{y^T Hy}$ 
12:   $x \leftarrow x_{\text{new}}$ 
13: end while
14:  $x^* \leftarrow x$ 
```

1.5.5 Uwagi końcowe

Metoda DFP znacząco przyspiesza zbieżność w porównaniu z metodami czysto gradientowymi, dzięki aproksymacji informacji o krzywiznie funkcji. Jej wadą może być mniejsza stabilność numeryczna w porównaniu z metodą BFGS, szczególnie dla funkcji źle uwarunkowanych.

1.6 Metoda BFGS (Broyden–Fletcher–Goldfarb–Shanno)

Metoda BFGS należy do klasy metod quasi-Newtona i jest jedną z najczęściej stosowanych metod optymalizacji bez ograniczeń. Jej celem jest minimalizacja funkcji

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}$$

z wykorzystaniem informacji o gradiencie, bez konieczności jawnego obliczania macierzy Hesjana. W porównaniu z metodą DFP, aktualizacja macierzy w BFGS charakteryzuje się lepszą stabilnością numeryczną.

1.6.1 Opis matematyczny algorytmu

Niech $x^{(k)} \in \mathbb{R}^2$ oznacza punkt w k -tej iteracji algorytmu, a gradient funkcji w tym punkcie:

$$g^{(k)} = \nabla f(x^{(k)}).$$

W metodzie BFGS przechowywana jest macierz H_k , będąca aproksymacją odwrotności Hesjana $(\nabla^2 f(x^{(k)}))^{-1}$. Kierunek poszukiwań wyznaczany jest jako:

$$d^{(k)} = -H_k g^{(k)}.$$

Po wykonaniu jednowymiarowej minimalizacji wzdłuż kierunku $d^{(k)}$ otrzymuje się nowy punkt:

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}.$$

Definiujemy wektory:

$$s_k = x^{(k+1)} - x^{(k)}, \quad y_k = g^{(k+1)} - g^{(k)}.$$

oraz skalar:

$$\rho_k = \frac{1}{y_k^T s_k}.$$

Aktualizacja macierzy H_k w metodzie BFGS dana jest wzorem:

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T.$$

Algorytm jest wykonywany iteracyjnie aż do spełnienia kryterium stopu:

$$\|\nabla f(x^{(k)})\| < \varepsilon.$$

1.6.2 Przykład liczbowy

Rozważmy funkcję:

$$f(x, y) = x^2 + 2y^2,$$

która posiada minimum globalne w punkcie $(0, 0)$.

Gradient funkcji:

$$\nabla f(x, y) = (2x, 4y).$$

Dane początkowe:

$$x^{(0)} = (1, 1), \quad H_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Iteracja 0. Gradient:

$$g^{(0)} = (2, 4).$$

Kierunek:

$$d^{(0)} = -H_0 g^{(0)} = (-2, -4).$$

Jednowymiarowa minimalizacja prowadzi do kroku:

$$\alpha_0 = \frac{5}{18}.$$

Nowy punkt:

$$x^{(1)} = (1, 1) + \frac{5}{18}(-2, -4) = \left(\frac{4}{9}, -\frac{1}{9}\right).$$

Gradient:

$$g^{(1)} = \left(\frac{8}{9}, -\frac{4}{9}\right).$$

Aktualizacja macierzy H_1 .

$$s_0 = \left(-\frac{5}{9}, -\frac{10}{9}\right), \quad y_0 = \left(-\frac{10}{9}, -\frac{40}{9}\right).$$

Ponieważ:

$$y_0^\top s_0 > 0,$$

aktualizacja BFGS zachowuje dodatnią określoność macierzy H_1 .

Po podstawieniu do wzoru BFGS otrzymujemy macierz H_1 , która w tym przypadku stanowi dokładną odwrotność Heszjana funkcji f .

Iteracja 1. Kierunek:

$$d^{(1)} = -H_1 g^{(1)}.$$

Jednowymiarowa minimalizacja wzdłuż tego kierunku prowadzi do punktu:

$$x^{(2)} = (0, 0).$$

Gradient:

$$g^{(2)} = (0, 0).$$

Ponieważ $\|\nabla f(x^{(2)})\| = 0$, spełnione jest kryterium stopu i algorytm zostaje zakończony.

1.6.3 Wynik końcowy

Metoda BFGS zakończyła działanie po dwóch iteracjach, zwracając punkt:

$$x^* = (0, 0),$$

który jest minimum globalnym analizowanej funkcji.

1.6.4 Pseudokod

Algorithm 6 Metoda BFGS (postać zwarta – aktualizacja Shermana–Morrisona)

```

1: Dane: funkcja  $f(x)$ , punkt startowy  $x_0$ , tolerancja  $\varepsilon$ 
2: Wynik: minimum  $x^*$ 
3:  $x \leftarrow x_0$ 
4:  $H \leftarrow I$  {początkowa aproksymacja odwrotności Hessianu}
5: while  $\|\nabla f(x)\| > \varepsilon$  do
6:    $d \leftarrow -H\nabla f(x)$ 
7:   Wyznacz krok  $\alpha$  (przeszukiwanie liniowe)
8:    $x_{\text{new}} \leftarrow x + \alpha d$ 
9:    $s \leftarrow x_{\text{new}} - x$ 
10:   $y \leftarrow \nabla f(x_{\text{new}}) - \nabla f(x)$ 
11:   $\rho \leftarrow \frac{1}{y^T s}$ 
12:   $H \leftarrow (I - \rho s y^T) H (I - \rho y s^T) + \rho s s^T$ 
13:   $x \leftarrow x_{\text{new}}$ 
14: end while
15:  $x^* \leftarrow x$ 

```

1.6.5 Uwagi końcowe

Metoda BFGS łączy szybkie tempo zbieżności z wysoką stabilnością numeryczną. W praktyce jest ona często preferowana względem metody DFP, szczególnie w przypadku funkcji źle uwarunkowanych lub problemów o większym wymiarze.

1.7 Algorytm ewolucyjny (GA)

Algorytm ewolucyjny (Genetic Algorithm, GA) jest niedeterministyczną, populacyjną metodą optymalizacji, inspirowaną mechanizmami biologicznej ewolucji. W przeciwieństwie do metod deterministycznych, algorytm GA operuje na zbiorze punktów (populacji), a jego działanie opiera się na procesach selekcji, krzyżowania oraz mutacji.

Algorytm ten jest szczególnie przydatny w przypadku funkcji:

- nieliniowych,
- niégładkich,
- posiadających wiele minimów lokalnych.

1.7.1 Opis matematyczny algorytmu

Niech:

$$P^{(k)} = \{x_1^{(k)}, x_2^{(k)}, \dots, x_N^{(k)}\} \subset \mathbb{R}^2$$

oznacza populację osobników w k -tej generacji, gdzie każdy osobnik $x_i^{(k)}$ jest wektorem zmiennych decyzyjnych.

Funkcja celu:

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}$$

pełni rolę funkcji przystosowania (fitness).

Jedna generacja algorytmu ewolucyjnego składa się z następujących etapów:

1. Ocena populacji:

$$F_i^{(k)} = f(x_i^{(k)}), \quad i = 1, \dots, N.$$

2. Selekcja — wybór najlepiej przystosowanych osobników do dalszej reprodukcji.

3. Krzyżowanie — tworzenie nowych osobników poprzez kombinację liniową genów:

$$x_{\text{child}} = \lambda x_a + (1 - \lambda)x_b, \quad \lambda \in (0, 1).$$

4. Mutacja — losowa perturbacja potomków:

$$x_{\text{mut}} = x_{\text{child}} + \xi, \quad \xi \sim \mathcal{U}(-\delta, \delta).$$

5. Utworzenie nowej populacji:

$$P^{(k+1)} = \{x_{\text{mut}}\}.$$

Algorytm jest powtarzany aż do spełnienia kryterium stopu, najczęściej w postaci osiągnięcia maksymalnej liczby generacji.

1.7.2 Przykład liczbowy

Rozważmy funkcję:

$$f(x, y) = (x - 1)^2 + (y - 2)^2,$$

która posiada minimum globalne w punkcie $(1, 2)$.

Zakres poszukiwań:

$$x \in [0, 4], \quad y \in [0, 4].$$

Dane algorytmu:

$$N = 4 \quad (\text{rozmiar populacji}), \quad G = 3 \quad (\text{liczba generacji}).$$

Generacja 0. Populacja początkowa (wylosowana):

Osobnik	(x, y)	$f(x, y)$
x_1	$(4, 4)$	13
x_2	$(0, 0)$	5
x_3	$(2, 3)$	2
x_4	$(3, 1)$	5

Najlepszy osobnik: $x_3 = (2, 3)$.

Generacja 1. Po selekcji wybieramy osobniki x_3 oraz x_2 . Krzyżowanie:

$$x_{\text{child}} = \frac{1}{2}(2, 3) + \frac{1}{2}(0, 0) = (1, 1.5).$$

Po mutacji otrzymujemy nową populację skupioną wokół punktu $(1, 2)$:

Osobnik	(x, y)	$f(x, y)$
x_1	$(1.0, 1.7)$	0.09
x_2	$(1.1, 1.6)$	0.17
x_3	$(0.9, 1.8)$	0.05
x_4	$(1.2, 1.7)$	0.13

Generacja 2. Dalsze krzyżowanie i mutacje prowadzą do populacji:

Osobnik	(x, y)	$f(x, y)$
x_1	$(1.0, 1.85)$	0.0225
x_2	$(0.95, 1.9)$	0.0125
x_3	$(1.05, 2.0)$	0.0025
x_4	$(1.0, 2.1)$	0.01

Najlepszy osobnik: $x_3 = (1.05, 2.0)$.

1.7.3 Kryterium zakończenia

Algorytm ewolucyjny kończy działanie po osiągnięciu maksymalnej liczby generacji:

$$k = G.$$

1.7.4 Wynik końcowy

Algorytm ewolucyjny zwrócił przybliżone minimum funkcji:

$$x^* \approx (1, 2),$$

co odpowiada minimum globalnemu analizowanej funkcji.

1.7.5 Pseudokod

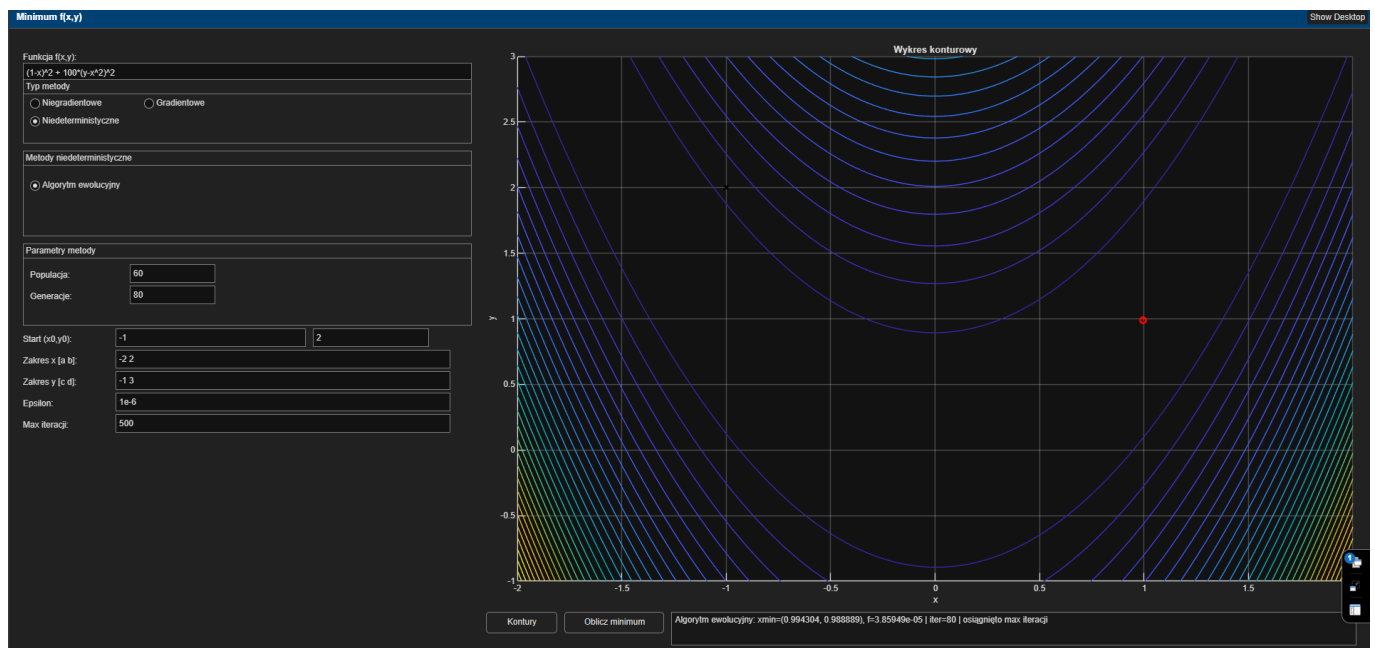
Algorithm 7 Algorytm genetyczny

- 1: **Dane:** funkcja celu $f(x)$, rozmiar populacji N , liczba generacji G
 - 2: **Wynik:** najlepszy osobnik x^*
 - 3: Zainicjalizuj populację P losowo
 - 4: **for** $g = 1, \dots, G$ **do**
 - 5: Oceń funkcję celu dla każdego osobnika w populacji P
 - 6: Wybierz osobniki do reprodukcji
 - 7: Zastosuj krzyżowanie
 - 8: Zastosuj mutację
 - 9: Utwórz nową populację P
 - 10: **end for**
 - 11: $x^* \leftarrow$ najlepszy osobnik z populacji
-

1.7.6 Uwagi końcowe

Algorytm ewolucyjny nie gwarantuje monotonicznej poprawy wartości funkcji w kolejnych iteracjach, jednak dzięki pracy na populacji punktów umożliwia skuteczne przeszukiwanie przestrzeni rozwiązań i redukuje ryzyko utknięcia w minimum lokalnym.

1.8 Instrukcja obsługi



Rysunek 1: Instrukcja obsługi

Aplikacja służy do wyznaczania minimum funkcji dwóch zmiennych $f(x, y)$ z możliwością wyboru metody optymalizacji, ustawienia parametrów oraz wizualizacji wyniku na wykresie konturowym.

1.8.1 Elementy interfejsu

Interfejs składa się z następujących sekcji:

1. **Pole *Funkcja* $f(x, y)$** – umożliwia wpisanie wzoru funkcji dwóch zmiennych w składni MATLAB. Przykład:

$$(1 - x)^2 + 100(y - x^2)^2.$$

Wzór musi być poprawnym wyrażeniem zależnym od zmiennych x oraz y .

2. **Sekcja *Typ metody*** – wybór kategorii algorytmu:

- *Niegradientowe*,
- *Gradientowe*,

- *Niedeterministyczne.*

Po zmianie kategorii wyświetla się odpowiednia lista metod, a pozostałe listy są ukrywane.

3. **Sekcja wyboru metody** – lista algorytmów dostępnych w danej kategorii:

- **Niegradientowe:** Hooke–Jeeves, Nelder–Mead, Powell,
- **Gradientowe:** Fletcher–Reeves, DFP, BFGS,
- **Niedeterministyczne:** Algorytm ewolucyjny (GA).

4. **Panel *Parametry metody*** – pojawia się dynamicznie i zawiera tylko te parametry, które są istotne dla wybranej metody, np.:

- Hooke–Jeeves: krok δ ,
- Fletcher–Reeves: krok początkowy α ,
- GA: rozmiar populacji oraz liczba generacji.

Jeżeli metoda nie wymaga dodatkowych parametrów, w panelu pojawia się komunikat o ich braku.

5. **Parametry ogólne** – wspólne dla wszystkich metod:

- **Start** (x_0, y_0) – punkt początkowy dla metod deterministycznych,
- **Zakres** $x \in [a, b]$ oraz $y \in [c, d]$ – przedziały wykorzystywane do rysowania konturów oraz jako ograniczenia w algorytmie GA,
- **Epsilon** – tolerancja zakończenia (dokładność),
- **Max iteracji** – maksymalny limit liczby iteracji (dla GA odpowiada maksymalnej liczbie generacji).

6. **Wykres konturowy** – przedstawia poziomice funkcji $f(x, y)$ na zadanym zakresie. Na wykresie oznaczane są:

- punkt startowy (x_0, y_0) jako znak **x**,
- wyznaczone minimum jako punkt (czerwone kółko).

7. **Pole wyniku** – tekstowe podsumowanie obliczeń w formacie:

metoda: xmin=(...), f=... | iter=... | powód zakończenia.

Powód zakończenia informuje, czy algorytm zakończył pracę z powodu spełnienia tolerancji czy przez osiągnięcie limitu iteracji.

1.8.2 Procedura użycia

Zalecany przebieg pracy z aplikacją:

1. Wprowadź wzór funkcji w polu **Funkcja f(x,y)**.
2. Ustaw **zakresy** $x \in [a, b]$ oraz $y \in [c, d]$ zgodnie z obszarem zainteresowania.
3. (Opcjonalnie) Ustaw punkt startowy (x_0, y_0) dla metod deterministycznych.
4. Wybierz **Typ metody** (niegradientowa / gradientowa / niedeterministyczna).
5. Wybierz konkretną metodę z listy.
6. Jeśli panel **Parametry metody** wyświetli dodatkowe pola, uzupełnij je.
7. Ustaw **Epsilon** (dokładność) oraz **Max iteracji** (limit obliczeń).
8. Kliknij **Kontury**, aby zaktualizować wykres konturowy (jeśli zmieniono funkcję lub zakresy).
9. Kliknij **Oblicz minimum**, aby uruchomić wybraną metodę optymalizacji.
10. Odczytaj wynik w polu tekstowym oraz z wykresu (zaznaczone minimum).

1.8.3 Uwagi praktyczne

- Dla funkcji o wielu minimach lokalnych wynik metod deterministycznych może zależeć od punktu startowego (x_0, y_0) . W takim przypadku zaleca się testowanie różnych punktów startowych oraz porównanie z algorytmem GA.
- Zwiększenie **Epsilon** przyspiesza obliczenia kosztem dokładności. Zmniejszenie **Epsilon** poprawia dokładność, ale może zwiększyć liczbę iteracji.
- Parametr **Max iteracji** zabezpiecza przed zbyt długim działaniem algorytmu w sytuacji braku zbieżności.
- W GA większa **Populacja** oraz większa liczba **Generacji** zwykle zwiększają szansę znalezienia lepszego minimum, kosztem czasu obliczeń.
- Jeżeli wykres konturowy jest “spłaszczony” lub trudno interpretowalny, warto zmienić zakresy $[a, b]$ i $[c, d]$ tak, aby obejmowały interesujący obszar funkcji.